Plano de Ação: Padronização de Código Python com Tox, Black, Isort e Flake8

Objetivo

Implementar um processo automatizado de padronização, formatação, análise estática e validação da qualidade do código Python para o projeto de automação de testes baseado em Selenium, Pytest, Behave e Guará Framework, garantindo:

- Conformidade com PEP8.
- Padronização automática do código.
- Organização consistente de imports.
- Detecção precoce de problemas de qualidade.
- Execução automatizada localmente e no GitHub Actions.
- Redução de falhas em Pull Requests.

FASE 1 - Levantamento da Situação Atual

Tarefa 1.1 – Identificar configuração existente

Verificar os seguintes arquivos:

```
pyproject.toml
pytest.ini
requirements.txt
```

Objetivo:

- Verificar se já existem configurações do:
  - black
  - isort
  - flake8
  - tox

Itens a conferir:

```
[tool.black]

[tool.isort]

[flake8]
```

```
6
7  [tox]
```

Documentar:

- Quais ferramentas já existem.
- Quais versões estão sendo utilizadas.
- Quais configurações precisam ser adicionadas.

Tarefa 1.2 – Mapear estrutura Python

Analisar os diretórios:

```
1  tests/
2  features/
```

Validar:

- Convenção de nomes de arquivos.
- Convenção de nomes de classes.
- Convenção de nomes de funções.
- Uso consistente de snake_case.

Identificar:

Exemplos incorretos:

```
1  LoginPage.py
2  loginPage.py
```

Padrão esperado:

```
1  login_page.py
```

FASE 2 - Configuração do Black

Tarefa 2.1 – Adicionar Black ao projeto

Adicionar ao requirements:

```
1  black
```

Ou criar:

```
1  requirements-dev.txt
```

Recomendado:

```
1  black>=24.0.0
```

Tarefa 2.2 – Configurar Black no pyproject.toml

Adicionar:

```
[tool.black]
line-length = 88
target-version = ["py311"]
include = '\.pyi?

Motivação:

* Compatível com Python 3.11.
* Padrão da comunidade.

 ***

 ## Tarefa 2.3 – Executar formatação inicial

 Executar:

 ```bash
 black .
```

Analisar:

- Arquivos modificados.
- Classes Page.
- Transactions.
- Fixtures.
- Steps do Behave.

Validar especialmente:

```
tests/pages/
tests/transactions/
features/steps/
```
```

FASE 3 - Configuração do Isort

Tarefa 3.1 – Instalar Isort

Adicionar:

```
isort>=5.13.0
```

Tarefa 3.2 – Configurar Isort

Adicionar no pyproject.toml:

```
[tool.isort]
profile = "black"
line_length = 88
multi_line_output = 3
include_trailing_comma = true
```

Tarefa 3.3 – Organizar imports existentes

Executar:

```
isort .
```

Validar separação correta:

```
# Biblioteca padrão
import os
from pathlib import Path

# Terceiros
import pytest
from selenium.webdriver.common.by import By

# Projeto
 from tests.pages.login_page import LoginPage
```

FASE 4 - Configuração do Flake8

Tarefa 4.1 – Instalar Flake8

Adicionar:

```
flake8>=7.0.0
```

Tarefa 4.2 – Configurar Flake8

Adicionar ao pyproject.toml:

```
[tool.flake8]
max-line-length = 88
extend-ignore = [
    "E203",
    "W503"
]
exclude = [
    ".git",
    "__pycache__",
     ".tox",
     "venv"
 ]
```

Essas regras evitam conflito com Black.

Tarefa 4.3 – Executar análise inicial

Executar:

```
flake8 .
```

Catalogar erros encontrados.

Classificar:

Tipo 1

Erros simples:

```
F401
E302
E305
```

Tipo 2

Problemas estruturais:

```
C901
F821
```

Criar backlog técnico.

FASE 5 - Configuração do Tox

Tarefa 5.1 – Instalar Tox

Adicionar:

```
tox>=4.0.0
```

Tarefa 5.2 – Criar arquivo tox.ini

Caso não exista:

```
[tox]
envlist = py311, lint

[testenv]
deps =
    -r requirements.txt

commands =
    pytest

 [testenv:lint]
 deps =
     black
     isort
     flake8

 commands =
     black --check .
     isort --check-only .
     flake8 .
```

Tarefa 5.3 – Testar execução

Executar:

```
tox
```

Validar:

- Ambiente py311.
- Ambiente lint.

Resultado esperado:

```
py311: OK
lint: OK
```

FASE 6 - Integração com GitHub Actions

Tarefa 6.1 – Criar workflow de qualidade

Criar:

```
.github/workflows/code-quality.yml
```

Tarefa 6.2 – Configurar pipeline

Estrutura sugerida:

```yaml
name: Code Quality

on:
  pull_request:
  push:
    branches:
      - main

jobs:
   lint:
     runs-on: ubuntu-latest

     steps:
       - uses: actions/checkout@v4

       - uses: actions/setup-python@v5
         with:
           python-version: "3.11"

       - run: pip install tox

```

```
22          - run: tox -e lint
```

Tarefa 6.3 – Validar workflow

Criar branch temporária.

Introduzir erro proposital:

```
1  import pytest
2  import os
```

Executar PR.

Confirmar que:

- isort falha.
- Pipeline bloqueia merge.

FASE 7 - Integração com Desenvolvimento Local

Tarefa 7.1 – Criar comandos padronizados

Documentar no README:

Formatar

```
1  black .
```

Ordenar imports

```
1  isort .
```

Verificar qualidade

```
1  flake8 .
```

Executar tudo

```
1  tox
```

Tarefa 7.2 – Criar atalhos no VSCode

Adicionar em:

```
1  .vscode/tasks.json
```

Tasks:

- Formatar código.
- Executar flake8.
- Executar tox.

FASE 8 - Documentação do Processo

Tarefa 8.1 – Atualizar documentação

Adicionar em:

```
docs/source/
```

Sugestão:

```
code_quality.md
```

Conteúdo:

- Objetivo.
- Ferramentas utilizadas.
- Como executar.
- Como corrigir falhas.
- Regras de revisão.

Tarefa 8.2 – Atualizar README

Criar seção:

```
## Qualidade de Código
```

Documentar:

```
tox
black .
isort .
flake8 .
```

FASE 9 - Critérios de Aceitação

A implementação será considerada concluída quando:

CA-01

Black formatar todo o projeto sem erros:

```
black .
```

Resultado:

```
All done!
```

CA-02

Isort não identificar imports desorganizados:

```
isort --check-only .
```

Resultado:

```
Success
```

CA-03

Flake8 retornar:

```
flake8 .
```

Sem erros críticos.

CA-04

Tox executar:

```
tox
```

Com sucesso.

CA-05

GitHub Actions bloquear PRs com violações.

CA-06

Documentação atualizada.

Melhoria Recomendada (Evolução Futura)

Após estabilizar Black, Isort e Flake8, incluir:

Pre-Commit

Arquivo:

```
.pre-commit-config.yaml
```

Ferramentas:

```
repos:
  - repo: https://github.com/psf/black
```

```
3     - repo: https://github.com/PyCQA/isort
4     - repo: https://github.com/PyCQA/flake8
```

Benefícios:

- Evita commits fora do padrão.
- Feedback imediato ao desenvolvedor.
- Reduz falhas no CI.

Sumário

O plano foi desenvolvido considerando a arquitetura atual do projeto (Pytest + Selenium + Behave + Guará Framework) e o objetivo de criar uma esteira de qualidade de código totalmente automatizada. A estratégia segue uma sequência incremental:

1. Diagnóstico da situação atual.
2. Padronização automática com Black.
3. Organização de imports com Isort.
4. Análise estática com Flake8.
5. Orquestração das validações via Tox.
6. Execução automática no GitHub Actions.
7. Padronização do uso pelos desenvolvedores.
8. Documentação do processo.
9. Evolução para Pre-Commit.

Dessa forma, a equipe garante qualidade contínua, reduz divergências de estilo em Pull Requests e mantém o framework de automação consistente ao longo do tempo.

Boas Práticas para Futuras Interações

Qualidade

- Centralizar todas as configurações em pyproject.toml.
- Evitar configurações duplicadas entre arquivos.

Automação

- Executar tox antes de abrir qualquer Pull Request.
- Tornar o workflow de lint obrigatório para merge.

Estrutura

- Manter Pages, Transactions e Fixtures seguindo o padrão snake_case.
- Utilizar imports absolutos do projeto.

Manutenibilidade

- Definir versões mínimas das ferramentas.
- Atualizar dependências periodicamente.

Evolução

- Adicionar mypy para validação de tipagem.
- Adicionar bandit para análise de segurança.
- Adicionar ruff como substituto futuro de parte das regras do flake8.
- Criar métricas de qualidade no pipeline (número de violações, cobertura de correções e tendência de evolução).

Esse conjunto de ações cria uma base sólida de governança técnica para o framework de automação de testes executado localmente e no GitHub Actions.

```
1
2  Motivação:
3
4  * Compatível com Python 3.11.
5  * Padrão da comunidade.
6
7  ***
8
9  ## Tarefa 2.3 – Executar formatação inicial
10
11  Executar:
12
13  ___CODE_BLOCK_PLACEHOLDER___9
14
15  Analisar:
16
17  * Arquivos modificados.
18  * Classes Page.
19  * Transactions.
20  * Fixtures.
21  * Steps do Behave.
22
23  Validar especialmente:
24
25  ___CODE_BLOCK_PLACEHOLDER___10
26
27  ***
```

```
28
29  # FASE 3 - Configuração do Isort
30
31  ## Tarefa 3.1 – Instalar Isort
32
33  Adicionar:
34
35  ___CODE_BLOCK_PLACEHOLDER___11
36
37  ***
38
39  ## Tarefa 3.2 – Configurar Isort
40
41  Adicionar no pyproject.toml:
42
43  ___CODE_BLOCK_PLACEHOLDER___12
44
45  ***
46
47  ## Tarefa 3.3 – Organizar imports existentes
48
49  Executar:
50
51  ___CODE_BLOCK_PLACEHOLDER___13
52
53  Validar separação correta:
54
55  ___CODE_BLOCK_PLACEHOLDER___14
56
57  ***
58
59  # FASE 4 - Configuração do Flake8
60
61  ## Tarefa 4.1 – Instalar Flake8
62
63  Adicionar:
64
65  ___CODE_BLOCK_PLACEHOLDER___15
66
67  ***
```

```
68
69  ## Tarefa 4.2 – Configurar Flake8
70
71  Adicionar ao pyproject.toml:
72
73  ___CODE_BLOCK_PLACEHOLDER___16
74
75  Essas regras evitam conflito com Black.
76
77  ***
78
79  ## Tarefa 4.3 – Executar análise inicial
80
81  Executar:
82
83  ___CODE_BLOCK_PLACEHOLDER___17
84
85  Catalogar erros encontrados.
86
87  Classificar:
88
89  ### Tipo 1
90
91  Erros simples:
92
93  ___CODE_BLOCK_PLACEHOLDER___18
94
95  ### Tipo 2
96
97  Problemas estruturais:
98
99  ___CODE_BLOCK_PLACEHOLDER___19
100
101  Criar backlog técnico.
102
103  ***
104
105  # FASE 5 - Configuração do Tox
106
107  ## Tarefa 5.1 – Instalar Tox
```

```
108
109 Adicionar:
110
111 ___CODE_BLOCK_PLACEHOLDER___20
112
113 ***
114
115 ## Tarefa 5.2 – Criar arquivo tox.ini
116
117 Caso não exista:
118
119 ___CODE_BLOCK_PLACEHOLDER___21
120
121 ***
122
123 ## Tarefa 5.3 – Testar execução
124
125 Executar:
126
127 ___CODE_BLOCK_PLACEHOLDER___22
128
129 Validar:
130
131 * Ambiente py311.
132 * Ambiente lint.
133
134 Resultado esperado:
135
136 ___CODE_BLOCK_PLACEHOLDER___23
137
138 ***
139
140 # FASE 6 - Integração com GitHub Actions
141
142 ## Tarefa 6.1 – Criar workflow de qualidade
143
144 Criar:
145
146 ___CODE_BLOCK_PLACEHOLDER___24
147
```

```
***

## Tarefa 6.2 – Configurar pipeline

Estrutura sugerida:

___CODE_BLOCK_PLACEHOLDER___25

***

## Tarefa 6.3 – Validar workflow

Criar branch temporária.

Introduzir erro proposital:

___CODE_BLOCK_PLACEHOLDER___26

Executar PR.

Confirmar que:

* isort falha.
* Pipeline bloqueia merge.

***

# FASE 7 - Integração com Desenvolvimento Local

## Tarefa 7.1 – Criar comandos padronizados

Documentar no README:

### Formatar

___CODE_BLOCK_PLACEHOLDER___27

### Ordenar imports

___CODE_BLOCK_PLACEHOLDER___28
```

```
188
189 ### Verificar qualidade
190
191 ___CODE_BLOCK_PLACEHOLDER___29
192
193 ### Executar tudo
194
195 ___CODE_BLOCK_PLACEHOLDER___30
196
197 ***
198
199 ## Tarefa 7.2 – Criar atalhos no VSCode
200
201 Adicionar em:
202
203 ___CODE_BLOCK_PLACEHOLDER___31
204
205 Tasks:
206
207 * Formatar código.
208 * Executar flake8.
209 * Executar tox.
210
211 ***
212
213 # FASE 8 - Documentação do Processo
214
215 ## Tarefa 8.1 – Atualizar documentação
216
217 Adicionar em:
218
219 ___CODE_BLOCK_PLACEHOLDER___32
220
221 Sugestão:
222
223 ___CODE_BLOCK_PLACEHOLDER___33
224
225 Conteúdo:
226
227 * Objetivo.
```

```
* Ferramentas utilizadas.
* Como executar.
* Como corrigir falhas.
* Regras de revisão.

***

## Tarefa 8.2 – Atualizar README

Criar seção:

___CODE_BLOCK_PLACEHOLDER___34

Documentar:

___CODE_BLOCK_PLACEHOLDER___35

***

# FASE 9 - Critérios de Aceitação

A implementação será considerada concluída quando:

### CA-01

Black formatar todo o projeto sem erros:

___CODE_BLOCK_PLACEHOLDER___36

Resultado:

___CODE_BLOCK_PLACEHOLDER___37

### CA-02

Isort não identificar imports desorganizados:

___CODE_BLOCK_PLACEHOLDER___38

Resultado:
```

```
___CODE_BLOCK_PLACEHOLDER___39

### CA-03

Flake8 retornar:

___CODE_BLOCK_PLACEHOLDER___40

Sem erros críticos.

### CA-04

Tox executar:

___CODE_BLOCK_PLACEHOLDER___41

Com sucesso.

### CA-05

GitHub Actions bloquear PRs com violações.

### CA-06

Documentação atualizada.

***

# Melhoria Recomendada (Evolução Futura)

Após estabilizar Black, Isort e Flake8, incluir:

### Pre-Commit

Arquivo:

___CODE_BLOCK_PLACEHOLDER___42

Ferramentas:
```

```
308
309  ___CODE_BLOCK_PLACEHOLDER___43
310
311  Benefícios:
312
313  * Evita commits fora do padrão.
314  * Feedback imediato ao desenvolvedor.
315  * Reduz falhas no CI.
316
317  ***
318
319  # Sumário
320
321  O plano foi desenvolvido considerando a arquitetura atual do
projeto (Pytest + Selenium + Behave + Guará Framework) e o
objetivo de criar uma esteira de qualidade de código totalmente
automatizada. A estratégia segue uma sequência incremental:
322
323  1. Diagnóstico da situação atual.
324  2. Padronização automática com Black.
325  3. Organização de imports com Isort.
326  4. Análise estática com Flake8.
327  5. Orquestração das validações via Tox.
328  6. Execução automática no GitHub Actions.
329  7. Padronização do uso pelos desenvolvedores.
330  8. Documentação do processo.
331  9. Evolução para Pre-Commit.
332
333  Dessa forma, a equipe garante qualidade contínua, reduz
divergências de estilo em Pull Requests e mantém o framework de
automação consistente ao longo do tempo.
334
335  # Boas Práticas para Futuras Interações
336
337  ### Qualidade
338
339  * Centralizar todas as configurações em
___CODE_BLOCK_PLACEHOLDER___44.
340  * Evitar configurações duplicadas entre arquivos.
341
342  ### Automação
```

```
343
344  * Executar ___CODE_BLOCK_PLACEHOLDER___45 antes de abrir
qualquer Pull Request.
345  * Tornar o workflow de lint obrigatório para merge.
346
347  ### Estrutura
348
349  * Manter Pages, Transactions e Fixtures seguindo o padrão
snake\_case.
350  * Utilizar imports absolutos do projeto.
351
352  ### Manutenibilidade
353
354  * Definir versões mínimas das ferramentas.
355  * Atualizar dependências periodicamente.
356
357  ### Evolução
358
359  * Adicionar ___CODE_BLOCK_PLACEHOLDER___46 para validação de
tipagem.
360  * Adicionar ___CODE_BLOCK_PLACEHOLDER___47 para análise de
segurança.
361  * Adicionar ___CODE_BLOCK_PLACEHOLDER___48 como substituto
futuro de parte das regras do flake8.
362  * Criar métricas de qualidade no pipeline (número de
violações, cobertura de correções e tendência de evolução).
363
364  Esse conjunto de ações cria uma base sólida de governança
técnica para o framework de automação de testes executado
localmente e no GitHub Actions.
```